Sistemas Distribuídos

Prof. Alexandre Sztajnberg 2023/1

Projeto – Desenvolvimento de um jogo de turno distribuído usando sockets e RMI (ou equivalente)

O objetivo deste projeto é desenvolver um pequeno jogo, o jogo da velha, e uma infraestrutura simples para o pareamento/match de dois jogadores para o jogo, jogo de turno. Essa infraestrutura vai ser tolerante a falhas.

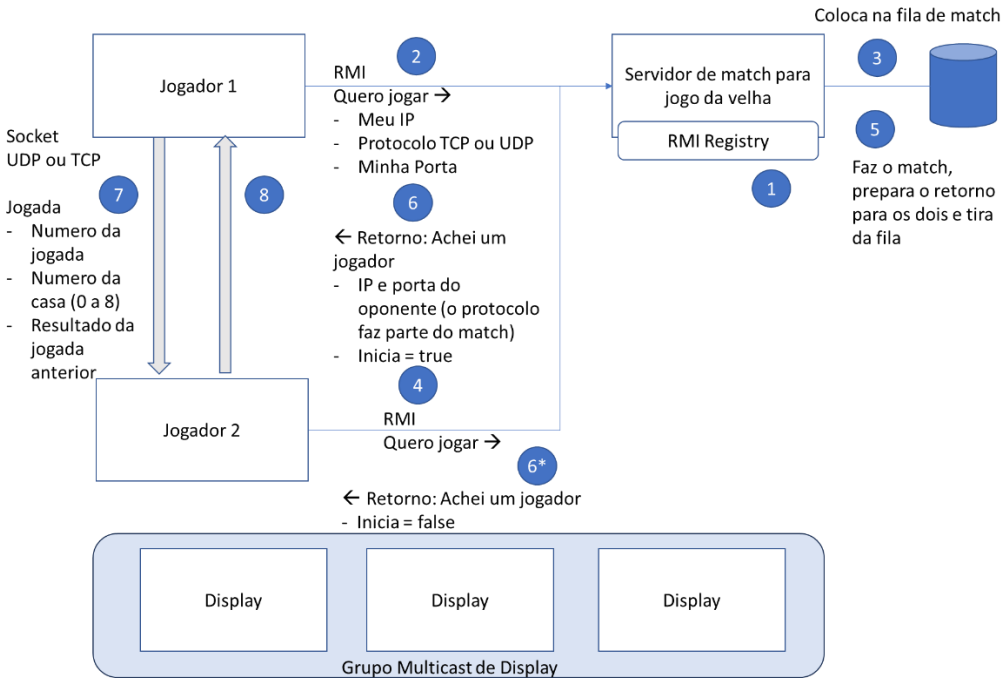


Figura 1. Arquitetura

O sistema/programa a ser implementado preferencialmente em Java, pois temos a biblioteca de sockets e o RMI, bem integrado e disponível. Mas, outras linguagens, como Python, podem ser utilizadas com as bibliotecas adequadas. Contate o professor para verificar.

O sistema desenvolvido deve executar no Linux, mais especificamente nas nossas máquinas virtuais 152.92.236.14, 236.16 e 236.17.

A documentação do sistema deve ser disponibilizada na conta da 236.14 através de uma página Web simples, mas organizada. Nessa página você também vai deixar um *pitch* de 10 min (ou o link para um *pitch*), explicando o jogo e dando alguns detalhes do código.

Vamos ter três módulos (programas):

- Um "servidor de match" que vai usar RMI

- O módulo jogador, que vai usar RMI para consultar o servidor de match, vai usar socket para fazer as jogadas com o outro jogador, e vai usar socket multicast para publicar as jogadas nos displays.
- O módulo display, que vai receber as jogadas por multicast e exibir. Podemos ter quantos displays quisermos e, ainda diferentes, desde que recebam as jogadas pelo mesmo canal multicast.

O servidor de match vai ficar em uma das 3 máquinas virtuais. Você escolhe. Nesta máquina o RMIRegistry também vai executar (pode usar o utilitário *rmiregistry* (ou rmid) mas o acionamento pelo próprio programa é muito mais prático).
Para não haver conflito de portas, você deve executar o *RMIRegistry* numa porta específica 98NN, ou de NN é o seu numero da conta (veja o exemplo disponível).

1. Servidor de Match

Quando um módulo cliente/jogador quer jogar, vai consultar o *servidor de match*. Para isso, registra o seu interesse em jogar informando o seu número IP, porta e protocolo (TCP ou UDP).

O servidor mantém uma lista com o registro de (outros) jogadores que também querem jogar. Se o servidor conseguir fazer um par (match) de dois jogadores, ele responde (*return* do método remoto) PARA OS DOIS JOGADORES. Para cada jogador é informado o número IP e a porta do seu oponente, e ainda, se o jogador é o que vai iniciar ou não. Um deles vai receber *true* e outro *false*. O match é feito apenas coincidindo o protocolo, já que não estamos pedindo a elaboração de vários jogos (isso seria relativamente simples, mas daria trabalho).

A resposta para os dois jogadores interessados é importante, por que finaliza a requisição para o par de jogadores procurando um oponente e também tira os dois jogadores da lista do servidor.

O Servidor de Match vai ter um método apenas. Este método recebe: o IP, a porta e o protocolo. Assinatura:

```
public json-ret getOponent (json-in)
```

O *json*-in terá a forma `{"ip": XXX.XXX.XXX.XXX, "porta": int, "protocolo": "udp"|"tcp"}`

O *json*-ret terá a forma `{"ip": XXX.XXX.XXX.XXX, "porta": int, "inicia": true|false}`

PS.: vamos definir como o json será representado. Talvez um array de char ou de byte.

O único aspecto mais elaborado é manter uma lista de requisições (ou seja, invocações do método *getOponent* pendentes, aguardando um match. Quando o match é obtido no servidor, deve ser dado o *return* para duas chamadas, pois o match é para um par de jogadores. Talvez seja necessário manter uma lista de chamadas ou threads. Busque sua solução.

Pense também como tratar time-outs. Ou seja, o que acontece seu um match demorar muito? Ou se o oponente desistir ... não estamos definindo uma solução para isso ...

2. Clientes / Jogadores

O módulo jogador será usado (i) para procurar um oponente e (ii) jogar uma partida apenas (poderíamos sofisticar, mas não vamos fazer isso desta vez).

O cliente terá duas "fases":

- Na primeira, invoca o método *getOponent* no servidor de match, remoto, com RMI, passa os parâmetros necessários e aguarda um retorno.
- Na segunda, passa a realizar jogadas até o jogo ser concluído, com uma vitória, derrota ou empate.
- A cada jogada, além de informar a jogada para o oponente, a jogada é enviada para publicação nos displays.

Match

Se o jogador recebeu o retorno do getOponent com "inicia" == true, então ele vai iniciar o jogo, transmitindo uma jogada ao jogador oponente. Em seguida aguarda uma resposta.

Na mesma linha, se o jogador recebeu o retorno do getOponent com "inicia" == false, então ele vai começar aguardando uma mensagem contendo a jogada do oponente.

Jogadas

As jogadas serão transmitidas enviadas para o oponente por *socket*. Vamos escolher UDP ou TCP. A escolha vai indicar como o seu código será feito. Cada aluno vai escolher um e fazer o módulo jogador com o protocolo escolhido. Não precisa fazer para os dois protocolos. Mas se você fizer, podemos estudar um bônus. Fale com o professor para decidir qual fazer.

Gostaríamos que o formato da mensagem fosse neutro. Assim vamos também usar o json para transmitir e receber as jogadas. Desta forma, vamos tentar fazer, depois, alguns testes de interoperabilidade das várias implementações.

Formado / interface da mensagem:

```
{ "Numero": int // número da jogada, com 9 casas, vamos ter [0..8]
  "Jogada": int // a casa a ser jogada [0..8]
  "Tabuleiro": int[9] // um array de 0 a 8
  "Resultado": "jog", "vit", "der", "emp"
}
```

Toda a logística, ou controle de sessão, fica ao seu encargo, como programador.

A cada jogada, o número da jogada é incrementado pelo jogador que realiza a jogada, começando por zero na primeira jogada. Use este número para controlar "cheat".

Uma jogada será simbolizada com o número da casa. Use esta informação para receber a jogada, colocar no seu tabuleiro local e verificar se é uma jogada legítima ou se o oponente

está jogando errado. Este controle você pode também fazer localmente, antes de jogar. Podemos pensar em um mecanismo de controle juntos.

Exemplo: ao receber uma jogada, e identificar que esta está errada, ao invés de fazer o seu movimento, o módulo automaticamente devolve a mesma mensagem, sem incrementar o número da jogada, obrigando o oponente corrigir a jogada e enviar a mesma novamente. Ao mesmo tempo, avisa para o seu "humano" que isso ocorreu.

Ao receber a mensagem com uma jogada, depois que identificou que está tudo correto, o módulo verifica se já chegamos em uma vitória ou um empate. No caso de vitória (do oponente, declara sua derrota para o seu "humano" e envia uma última mensagem para o oponente aceitando o resultado final.

Depois disso, o jogo acaba. O socket deve ser fechado (ou a conexão e o socket).

Display

Também a cada jogada, uma mensagem por socket **multicast** é enviada para o grupo de multicast dos módulos Display. O endereço multicast deve ser 225.1.1.NN e a porta multicast deve ser 88NN. Assim não teremos conflitos.

A mesma mensagem JSON (ou mais simplificada) deve ser enviada para o display, que vai exibir o "tabuleiro" e as jogadas (não precisa sofisticar, pode ser texto mesmo ...). Como se fossem canais de esporte ...

E o tempo para fazer isso tudo?

Você tem um mês para fazer esse trabalho. Basicamente é um trabalho de integração, já que você tem os exemplos. Numa próxima versão do enunciado podemos destacar os itens a serem avaliados. Obviamente, o funcionamento correto e o atendimento aos requisitos é importante.